18. Project Layout: Libraries vs Services

18. Project Layout: Libraries vs Services

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1 day
Slug: `18-project-layout`

Learning objectives

- Choose layouts for libraries vs services
- Use cmd/, internal/, pkg/ judiciously
- Keep main thin — logic in packages
- Avoid premature micro-packages

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Common service layout

```
cmd/api/main.go      # wiring only
internal/server/     # private app code
internal/store/
go.mod
```

Libraries

Export a stable API from the module root or clear subpackages. Semantic import versioning: `/v2` path for major version 2+.

Guidelines

- `internal/` for code you do not want strangers to import
- Prefer fewer packages until boundaries hurt
- Config via env/flags at main; pure functions underneath for tests

Common mistakes (especially from Python instincts)

- Copying a giant layout template for a 200-line tool
- Business logic inside main.go
- Exporting everything from pkg/ without need

Exercises

Work in `exercises/18-project-layout/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/18-project-layout/` until you have an honest failing attempt.

Self-check

1. What belongs in cmd/?
2. Why internal/?
3. When introduce a new package?

Further reading

- https://go.dev/doc/modules/layout
- https://github.com/golang-standards/project-layout